

Convolutions for Hands-on Computer Vision

All the **credit** goes to:

Brian Plancher, Harvard John A.
Paulson School of Engineering and
Applied Sciences, Barnard College,
Columbia University

brianplancher.com

Original video:

https://www.youtube.com/watch?v=Grf_21CXgo4

Computer Vision is Hard

**What color are the
pants and the shirt?**



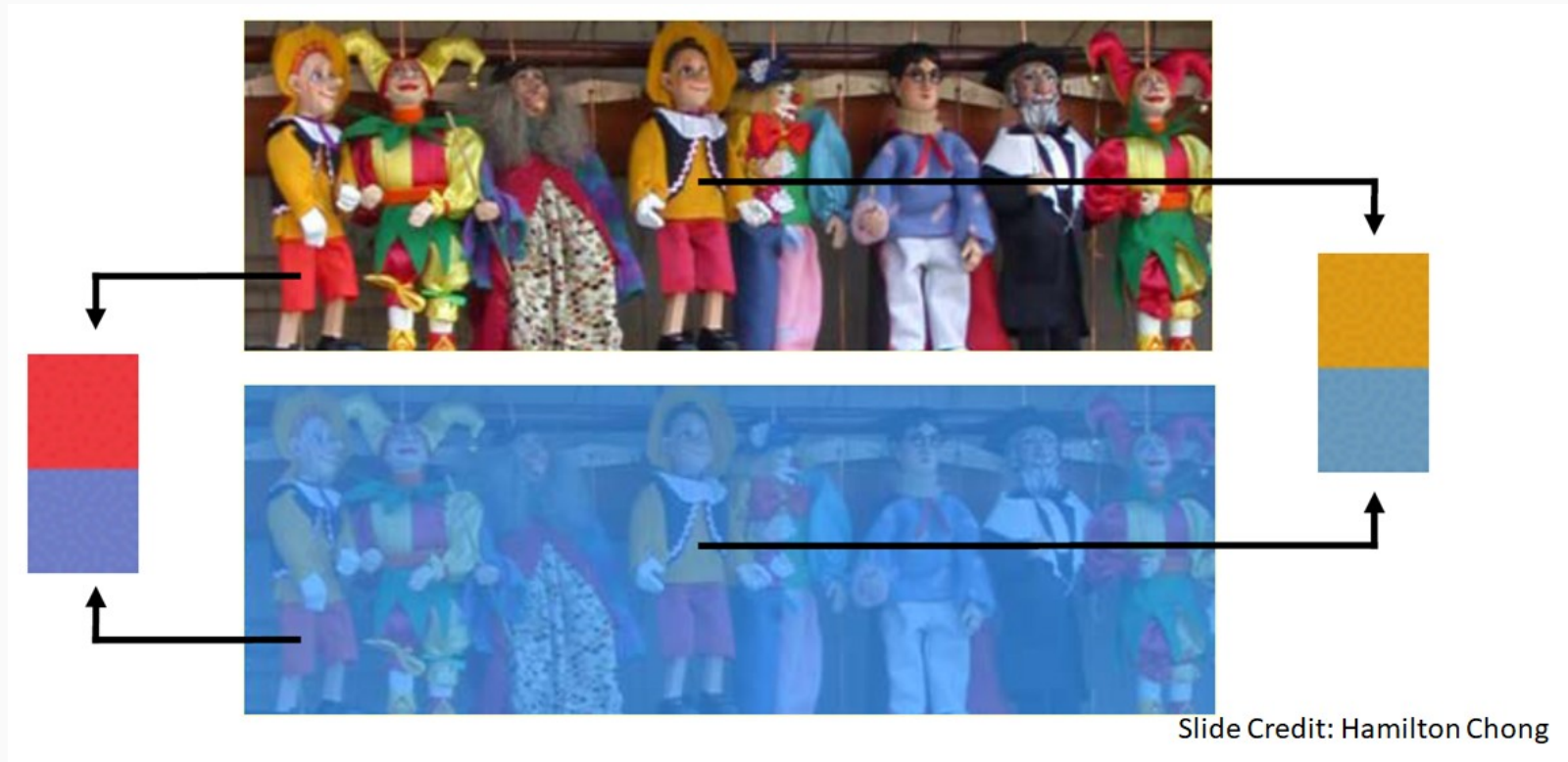
Slide Credit: Hamilton Chong

Computer Vision is Hard

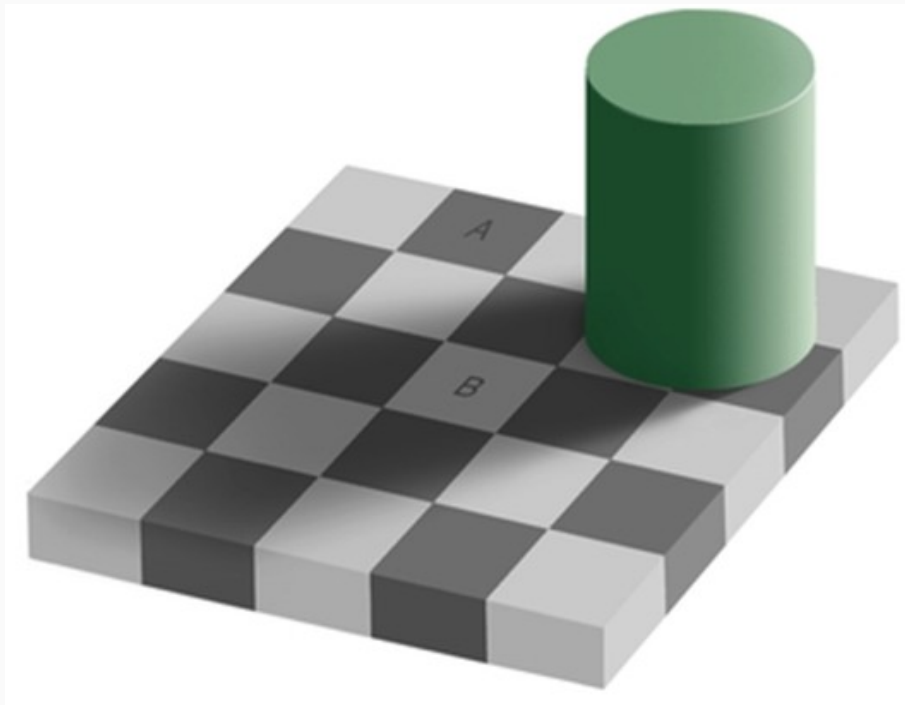


Slide Credit: Hamilton Chong

Computer Vision is Hard

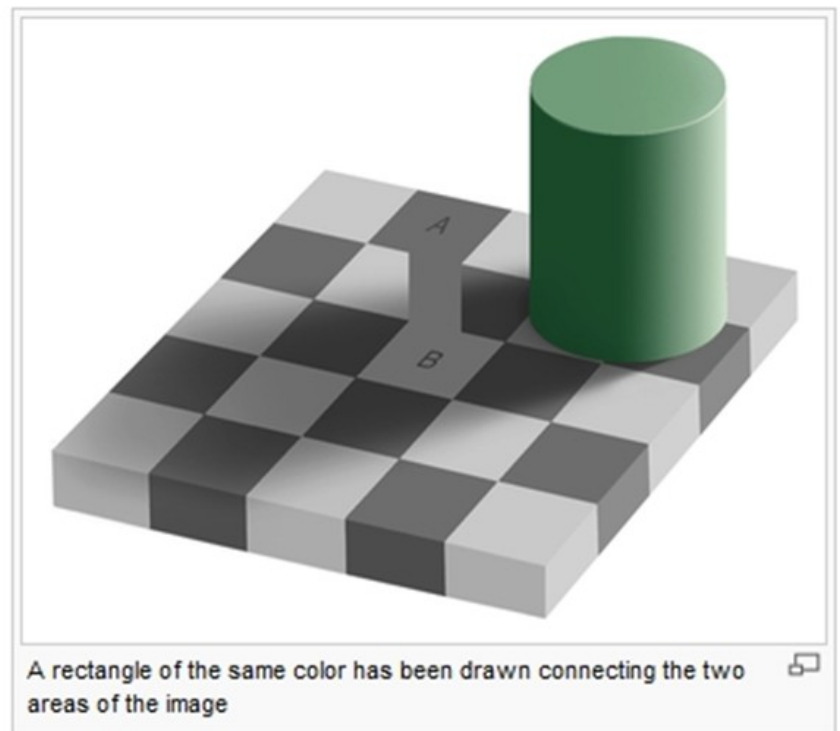
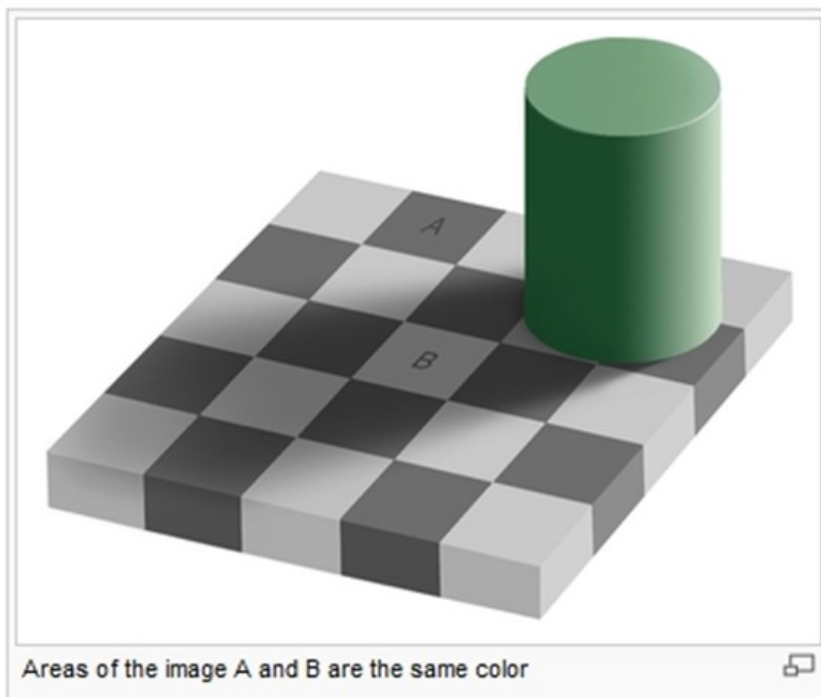


Computer Vision is Hard



Is square
A or B
darker in
color?

Computer Vision is Hard



What Features of the image might be important for self driving cars?



What Features of the image might be important for self driving cars?

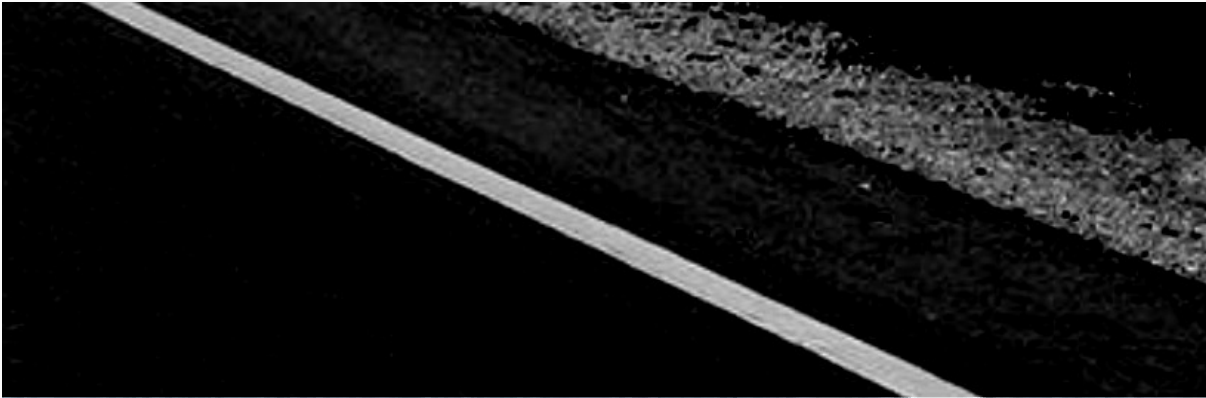


Maybe
straight
lines to
see the
lanes
of the
road?

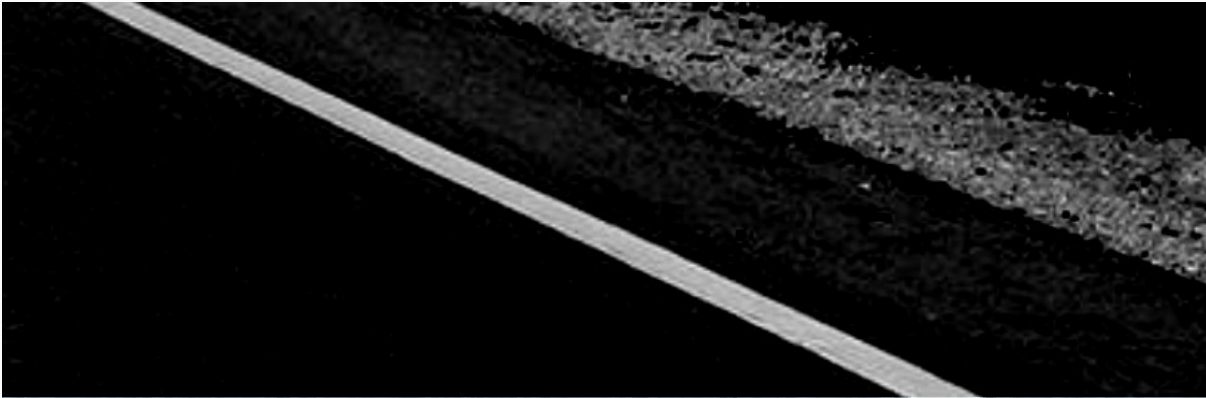
How might we find these features?



How might we find these features?



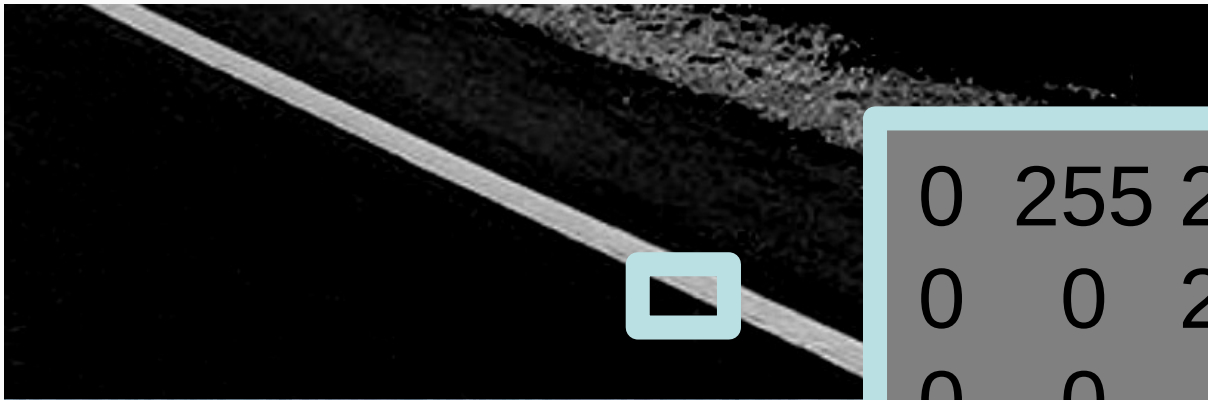
How might we find these features?



Black: 0

White: 255

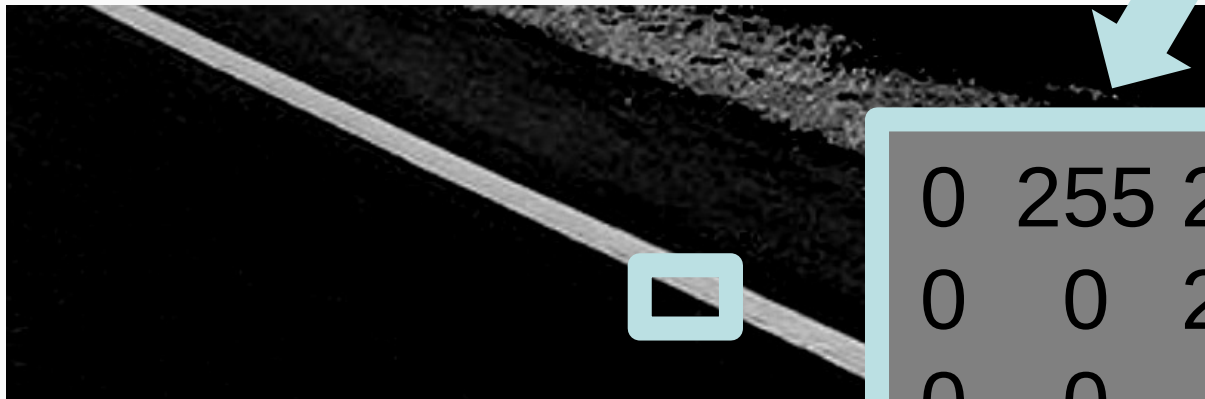
How might we find these features?



Black: 0
White: 255

0	255	255	255	255
0	0	255	200	255
0	0	0	255	255
0	0	0	0	255
0	0	0	0	0

How might we find these features?



Look for a Big Change!

Black: 0
White: 255

0	255	255	255	255
0	0	255	200	255
0	0	0	255	255
0	0	0	0	255
0	0	0	0	0

How might we find these features? Convolutions

How might we find these features? Convolutions

Original Image

0	0	0	255	255	255
0	0	0	255	255	255
0	0	0	255	255	255
0	0	0	255	255	255
0	0	0	255	255	255
0	0	0	255	255	255

How might we find these features? Convolutions

Original Image

0	0	0	255	255	255
0	0	0	255	255	255
0	0	0	255	255	255
0	0	0	255	255	255
0	0	0	255	255	255
0	0	0	255	255	255

Filter

-1	0	1
-1	0	1
-1	0	1

How might we find these features? Convolutions

Original Image

0	0	0	255	255	255
0	0	0	255	255	255
0	0	0	255	255	255
0	0	0	255	255	255
0	0	0	255	255	255
0	0	0	255	255	255

Filter

-1	0	1
-1	0	1
-1	0	1

How might we find these features? Convolutions

Original Image

0	0	0	255	255	255
0	0	0	255	255	255
0	0	0	255	255	255
0	0	0	255	255	255
0	0	0	255	255	255
0	0	0	255	255	255

Filter

-1	0	1
-1	0	1
-1	0	1

Output
Feature Map

765

How might we find these features? Convolutions

Original Image

0	0	0	255	255	255
0	0	0	255	255	255
0	0	0	255	255	255
0	0	0	255	255	255
0	0	0	255	255	255
0	0	0	255	255	255

Filter

-1	0	1
-1	0	1
-1	0	1

Output
Feature Map

0	765	765	0
0	765	765	0
0	765	765	0
0	765	765	0

How might we find these features? Convolutions

[Colab Link](#)

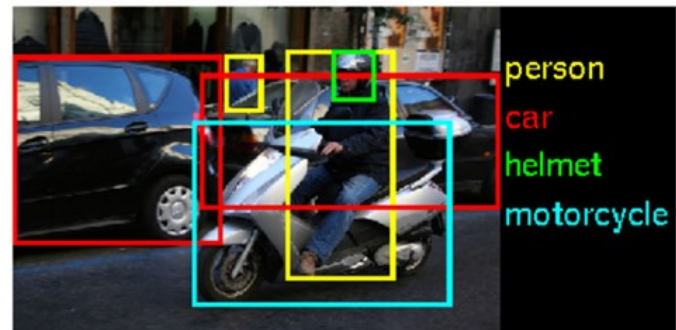
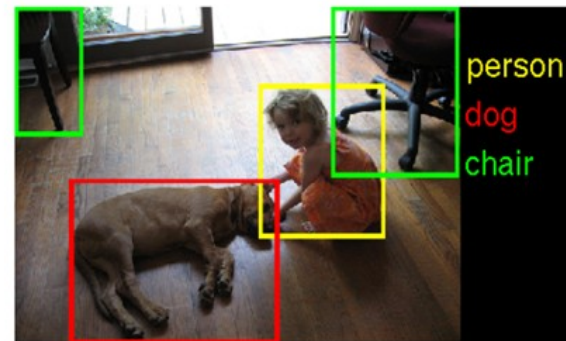
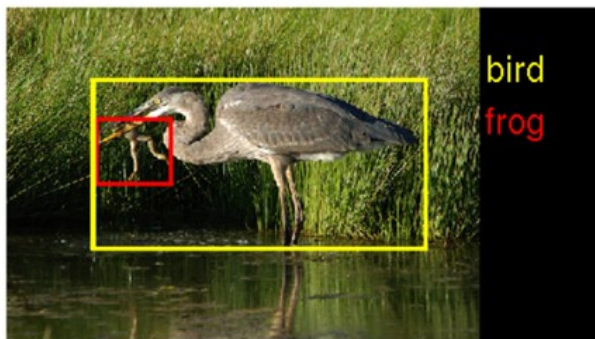


-1	0	1
-2	0	2
-1	0	1

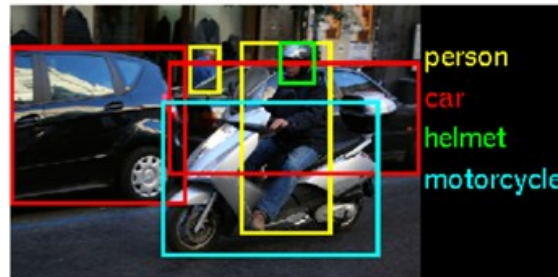
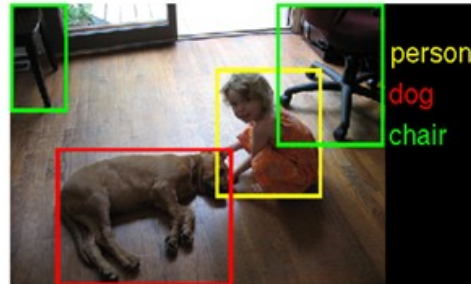
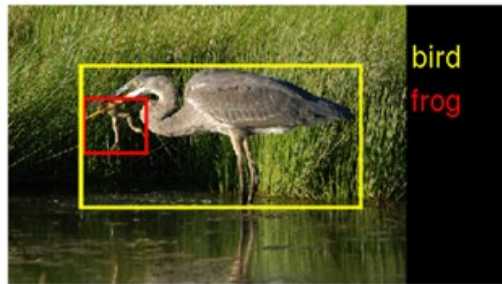


<https://setosa.io/ev/image-kernels/>

What features are needed for Object Detection?

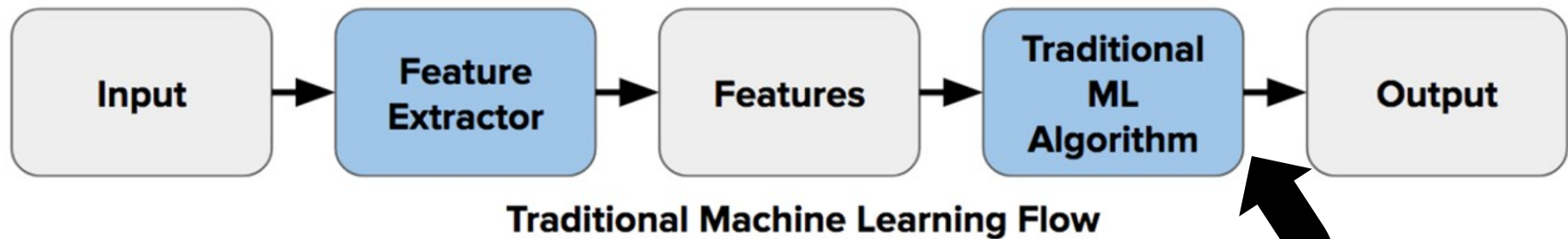


What features are needed for Object Detection?



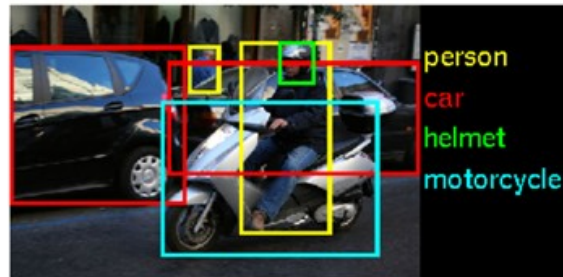
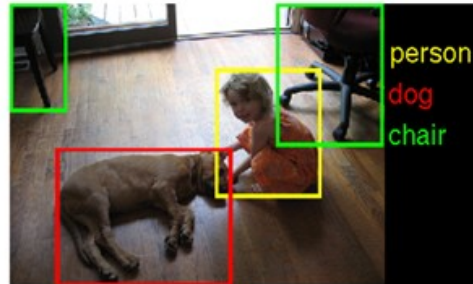
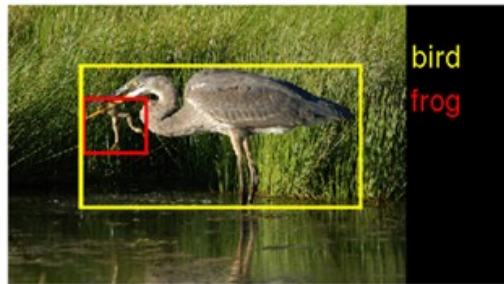
The ImageNet Challenge provided 1.2 million examples of 1,000 **labeled** items and challenged algorithms to learn from the data and then was tested on another 100,000 images

What features are needed for Object Detection?



Regression, Clustering, etc.

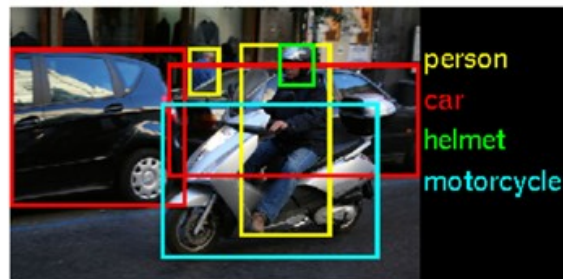
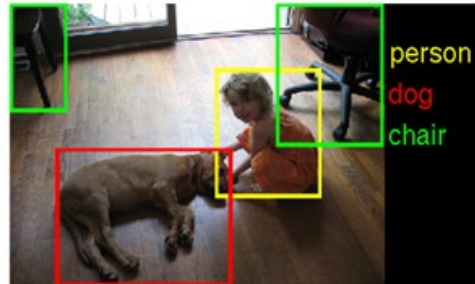
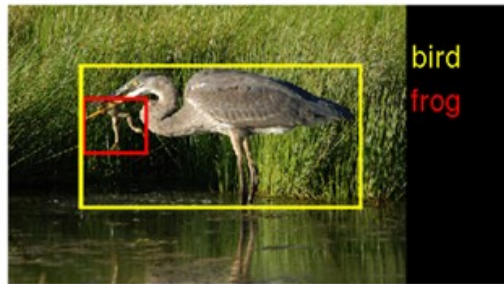
What features are needed for Object Detection?



In 2010 teams had
75-50% error

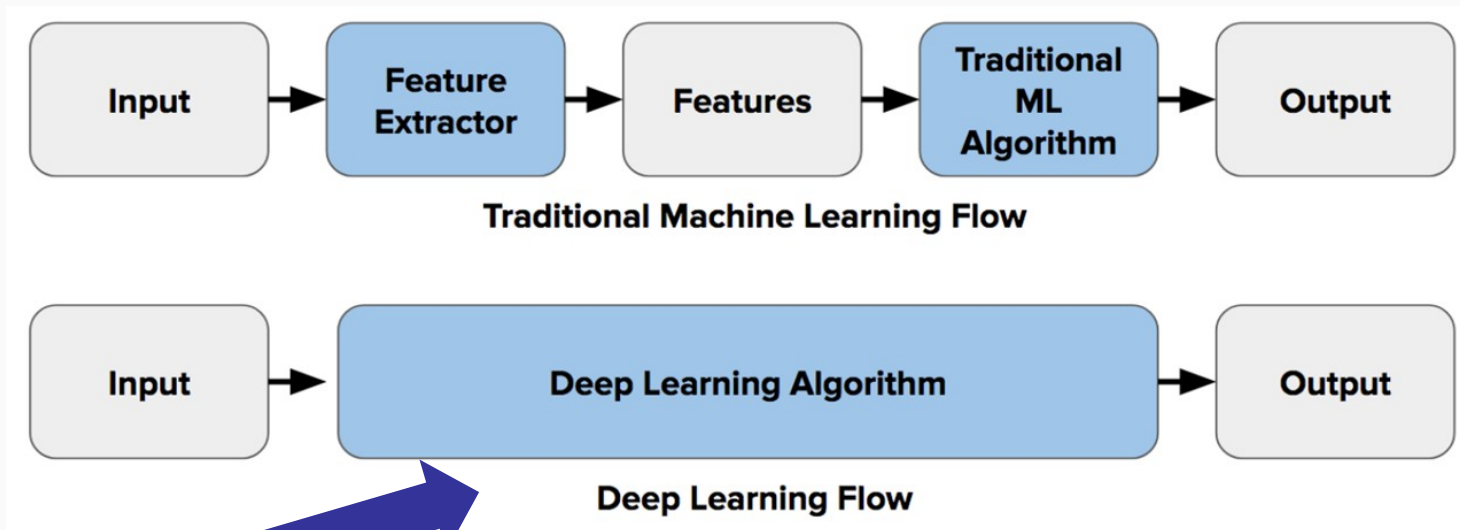
In 2011 teams had
75-25% error

What features are needed for Object Detection?



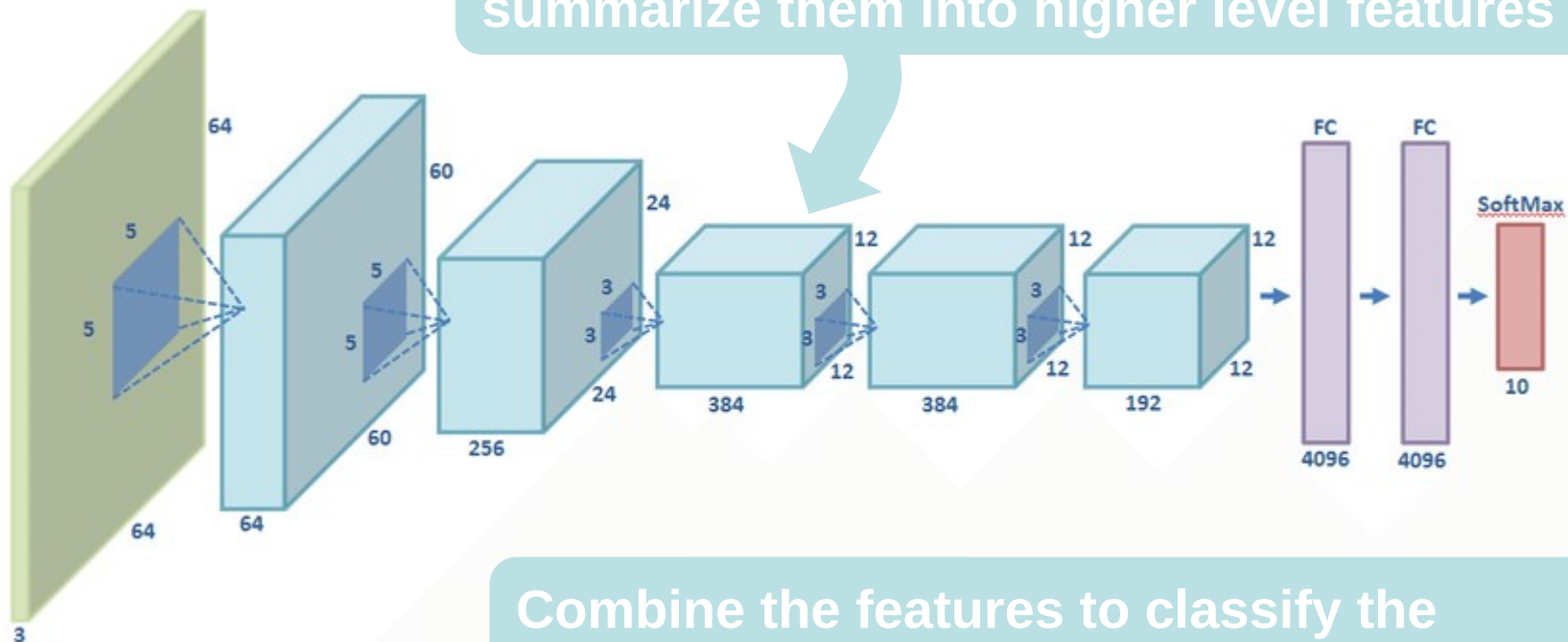
In 2012 still no team
had less than 25%
error barrier except
AlexNet at 15%

What features are needed for Object Detection?



Let the computer figure out its own features and how to combine them!

Use convolutions to find features and the summarize them into higher level features



Combine the features to classify the various objects in the dataset

How might we find these features? Convolutions

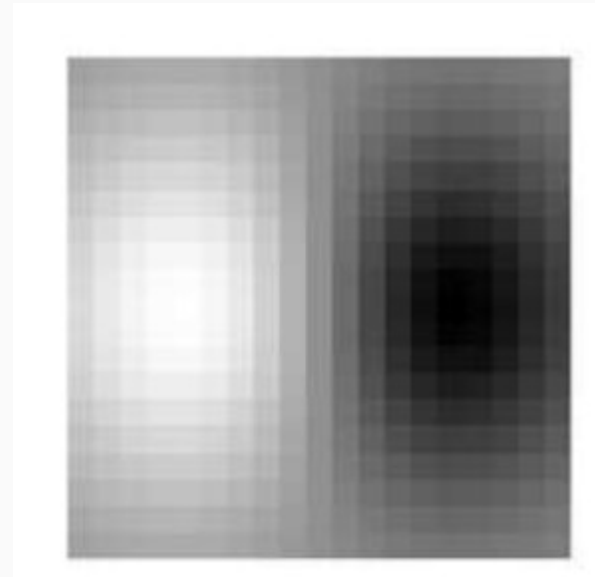
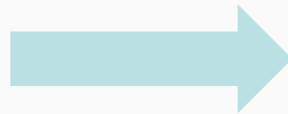


-1	0	1
-2	0	2
-1	0	1

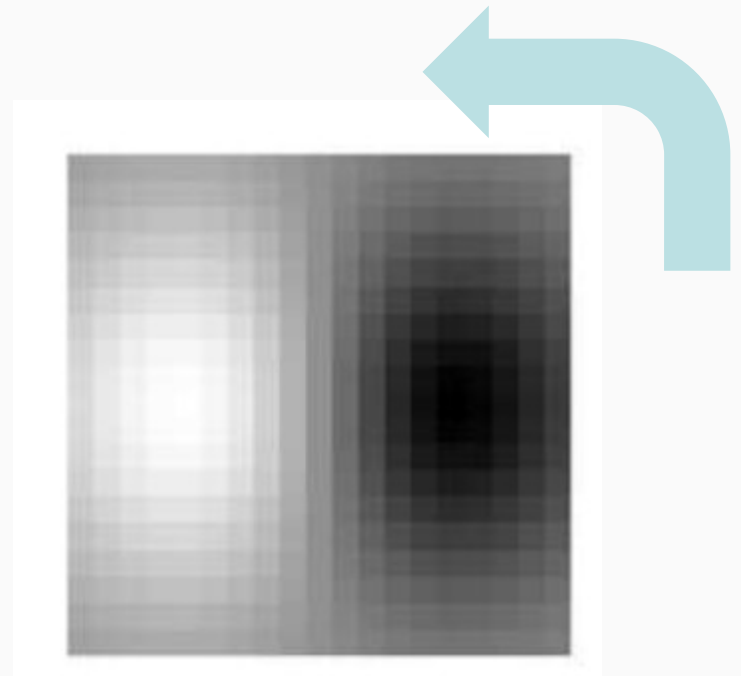


How might we find these features? Convolutions

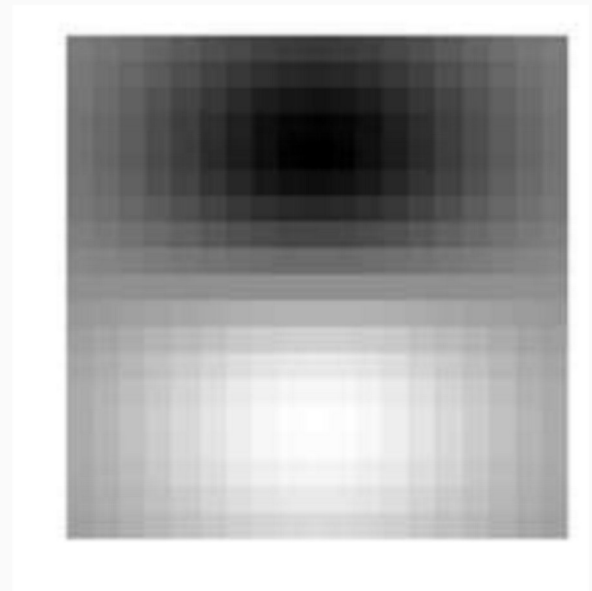
-1	0	1
-2	0	2
-1	0	1



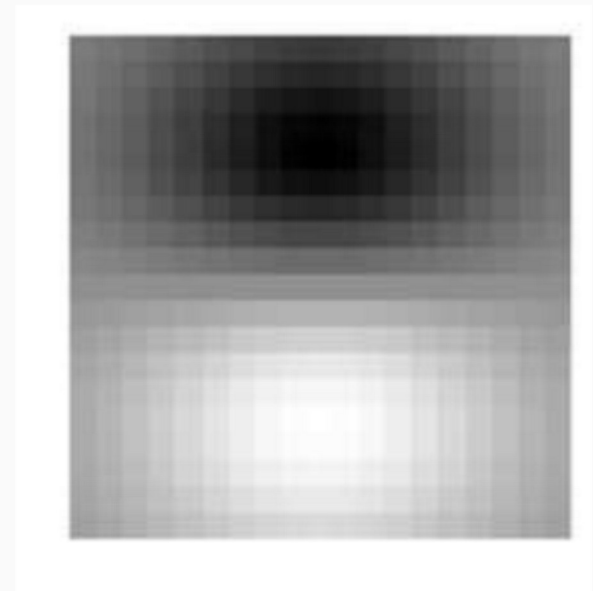
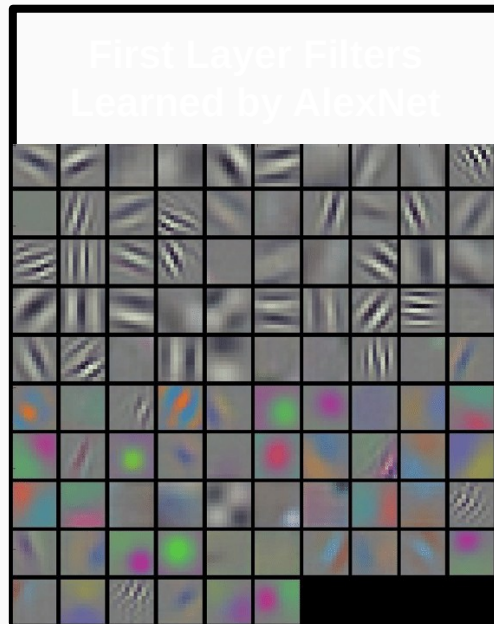
How might we find these features? Convolutions



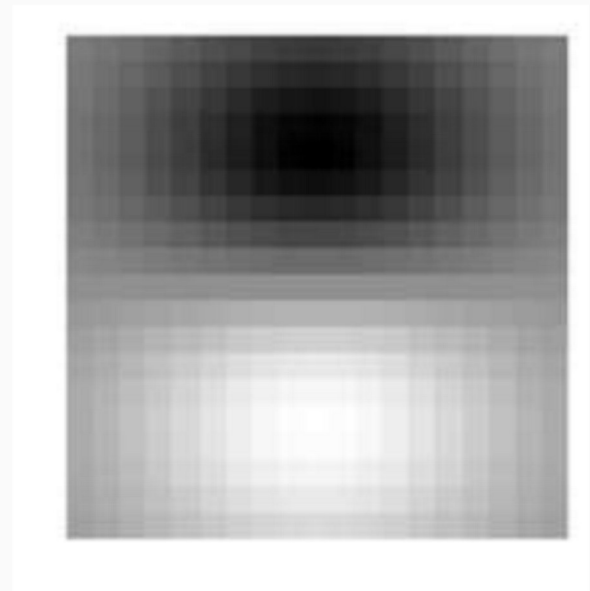
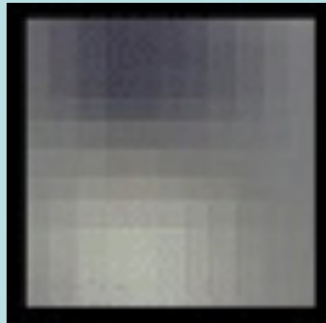
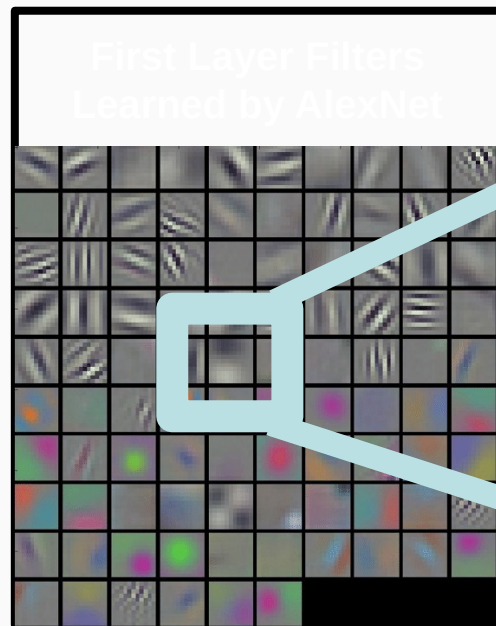
How might we find these features? Convolutions



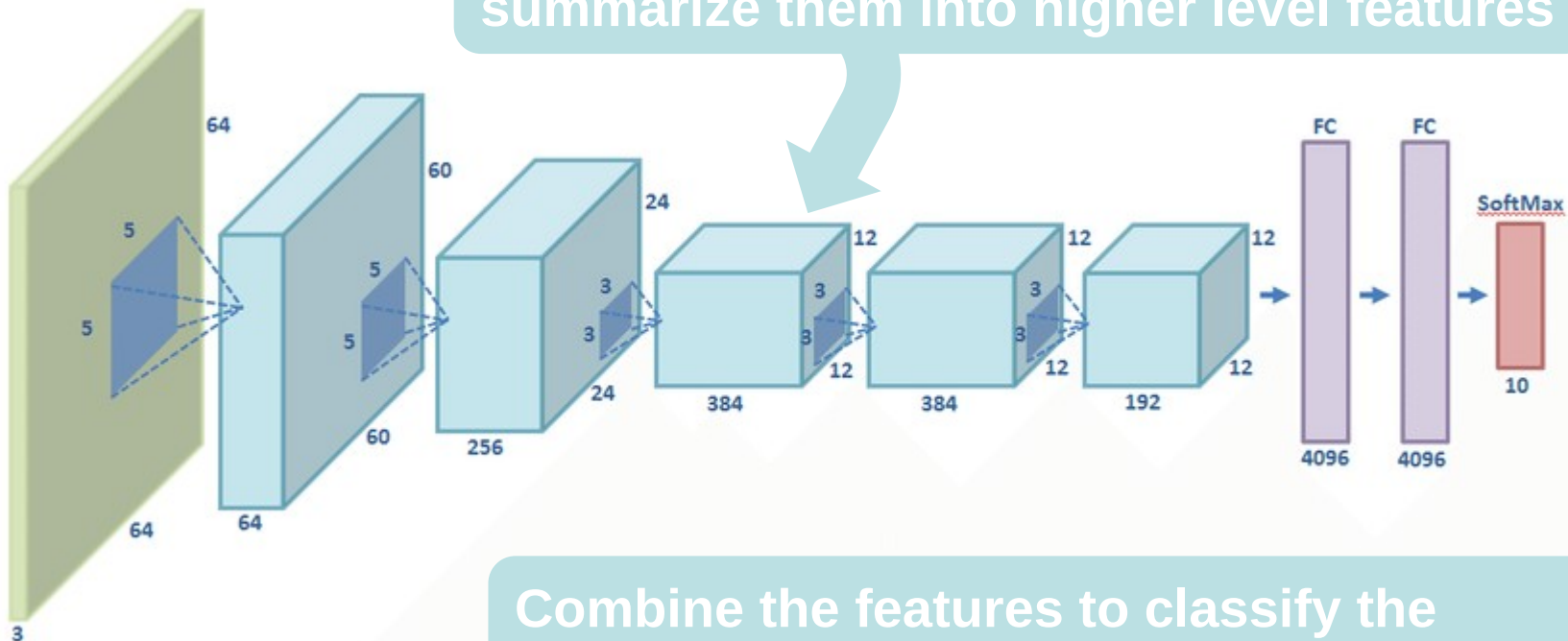
How might we find these features? Convolutions



How might we find these features? Convolutions

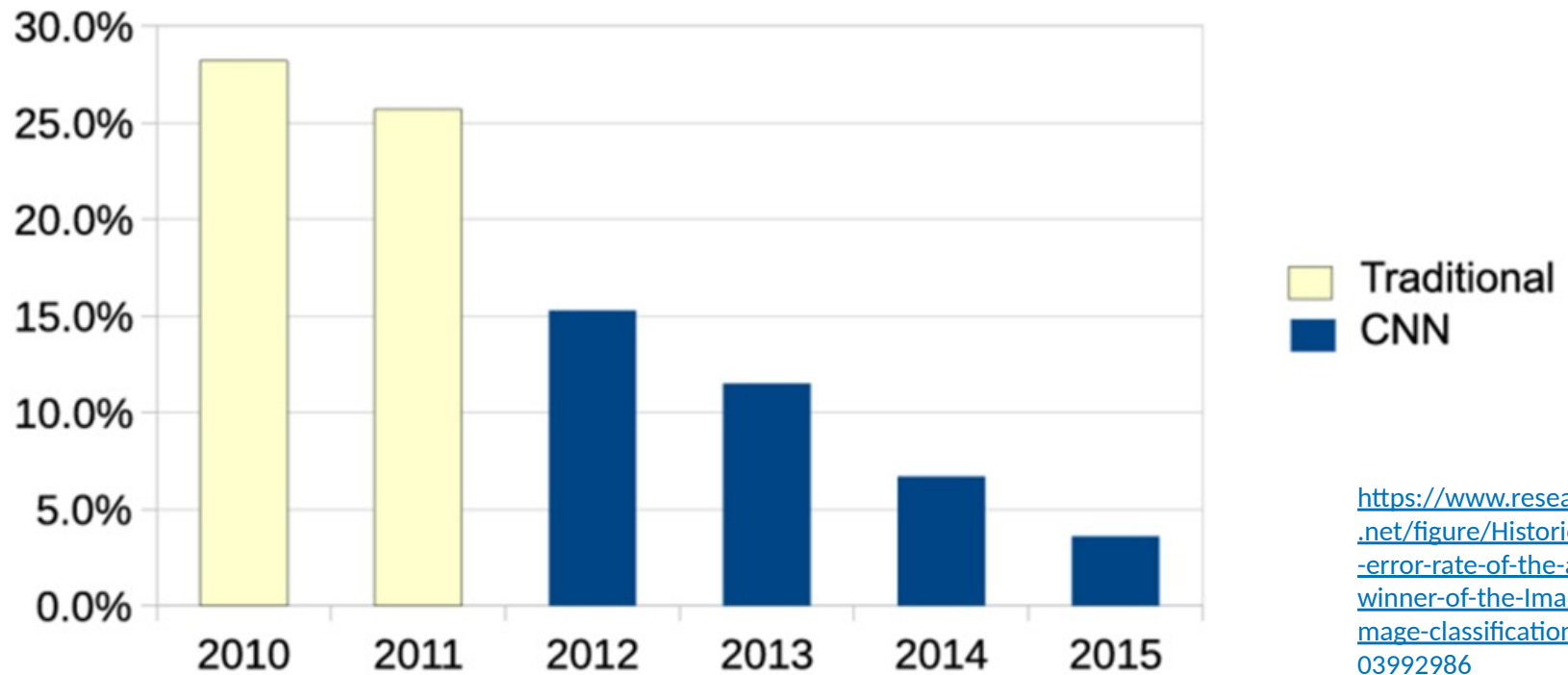


Use convolutions to find features and the summarize them into higher level features



Combine the features to classify the various objects in the dataset

What features are needed for Object Detection?



A word of caution...

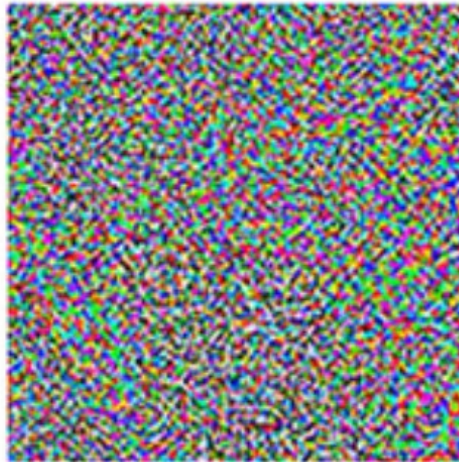
Ackerman "Hacking the Brain With Adversarial Images"



"panda"

57.7% confidence

+ ϵ



=

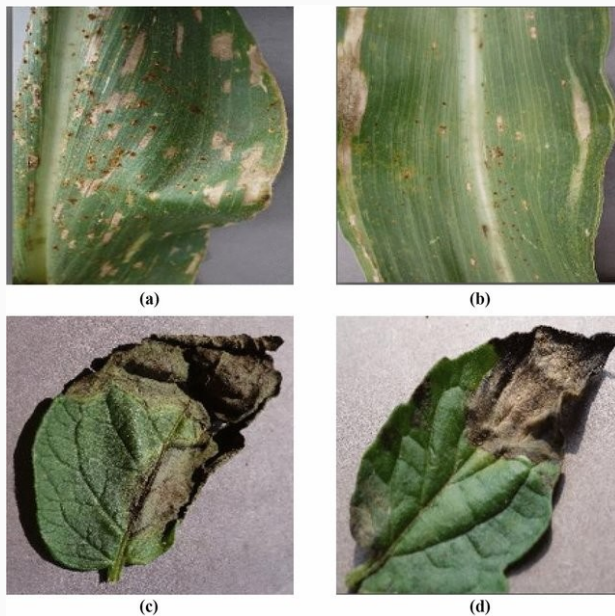


"gibbon"

99.3% confidence

There is **no model** of
the world semantically
just mathematically

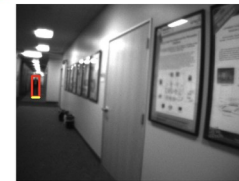
- Let's get started with the first TinyML Lab!
- We will work on Image Classification



tinyML Vision supports human detection cases



Half body



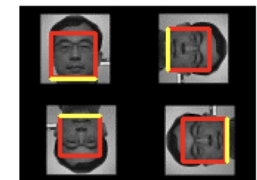
Full body



3/4 body



Change Detection



Multiple face orientation

Qualcomm



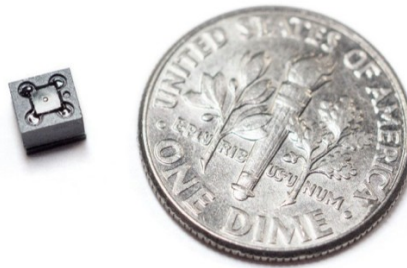
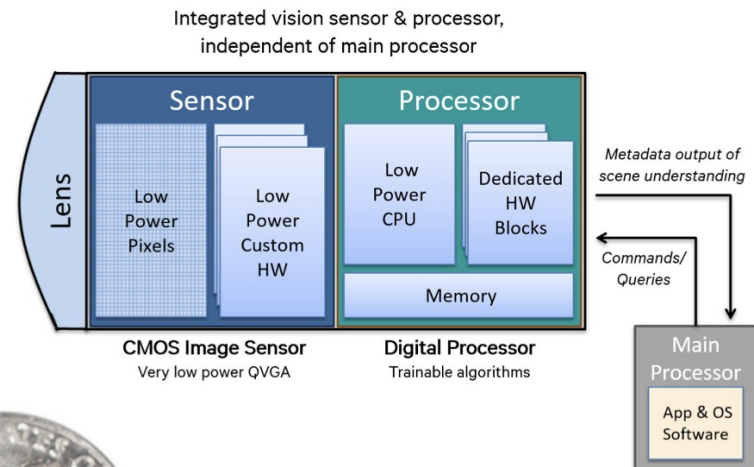
Credit: Evgeni Gousev, tinyML Foundation

tinyML for Always-On Vision

Qualcomm always-on computer vision module

Key features:

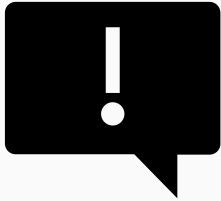
- Ultra-low power, < 1 mW (end-to-end)
- Small size
- Privacy (output is metadata)
- Configurable for different use cases
- QVGA sensor, Near-IR compatible
- Low cost



Qualcomm



Credit: Evgeni Gousev, tinyML Foundation



It is better to gather data with the same system that will then run the inference. With image we can also use a smartphone or a PC, but it is better to use the Nicla Vision itself!

Embedded Machine Learning (TinyML) Workflow Review

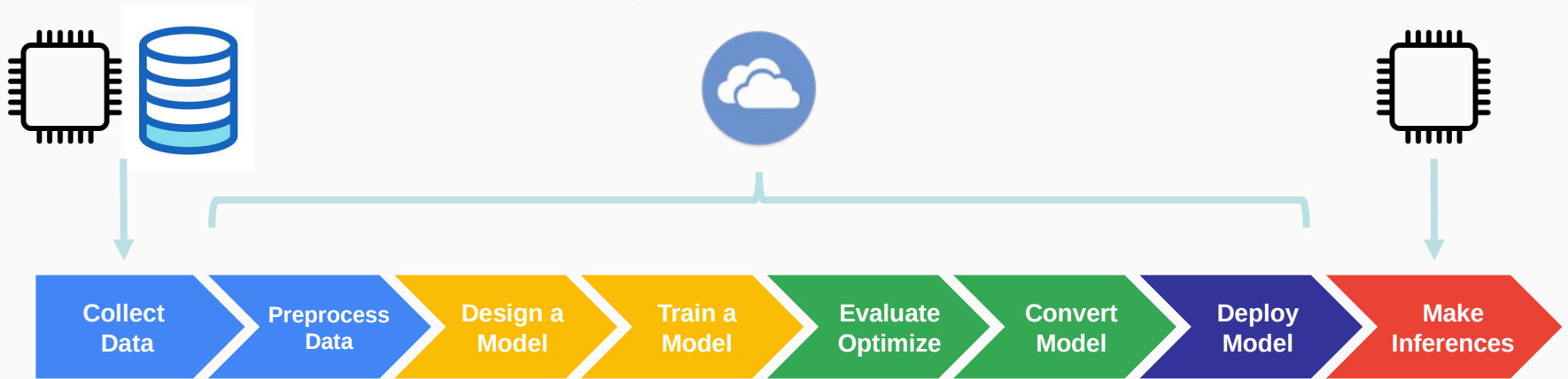
Machine Learning Workflow



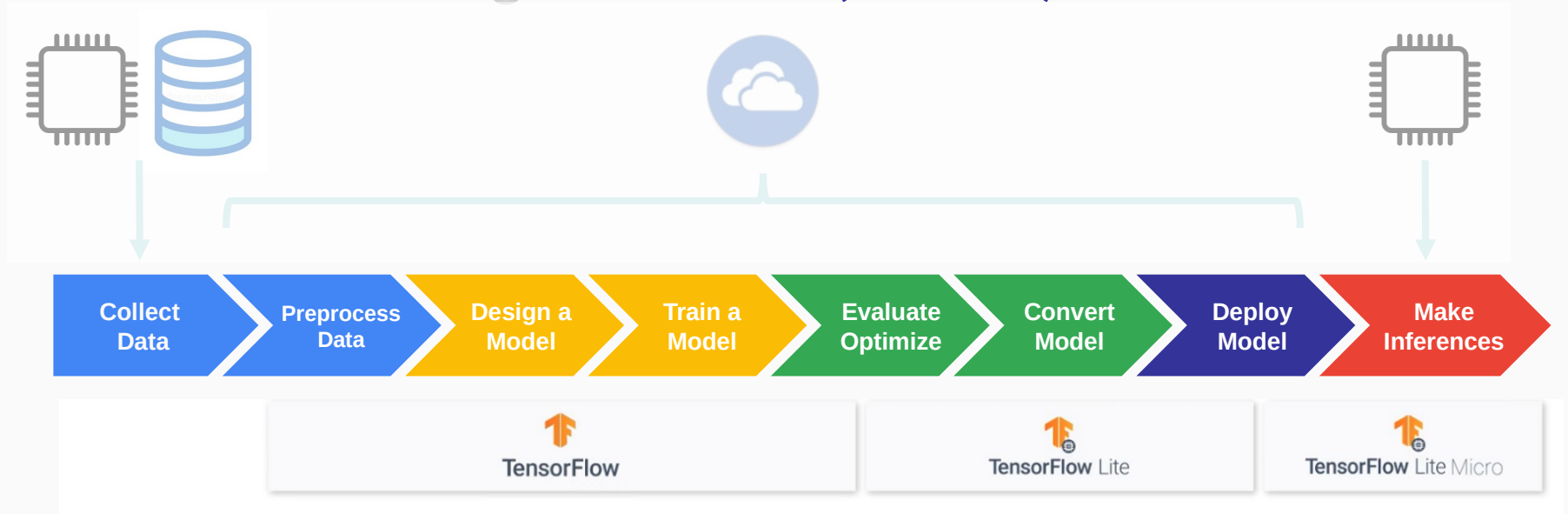
Tiny Machine Learning Workflow (“What”)



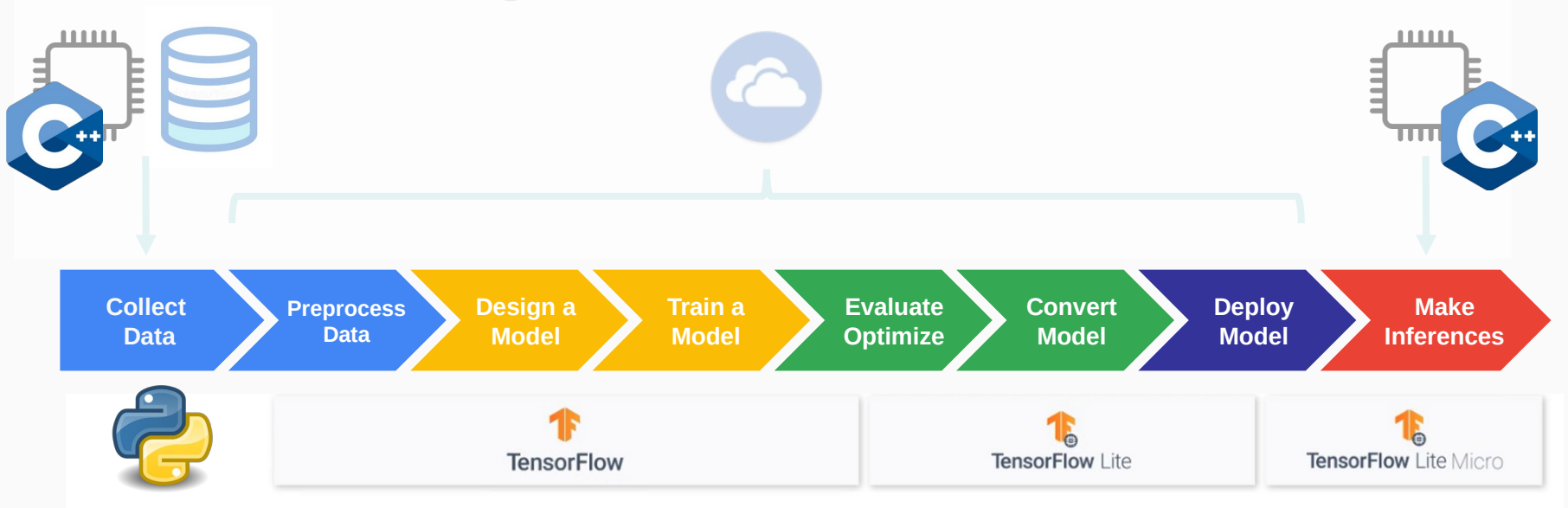
Tiny Machine Learning Workflow (“Where”)



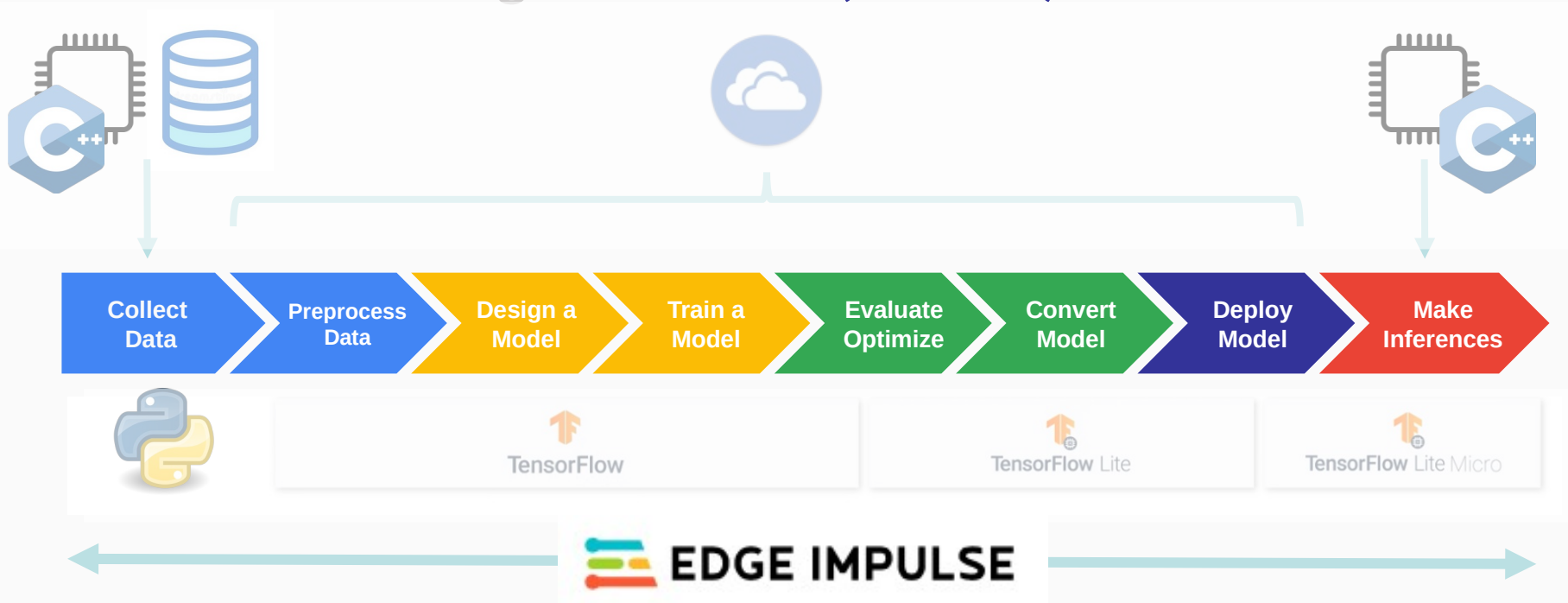
Machine Learning Workflow (“How”)

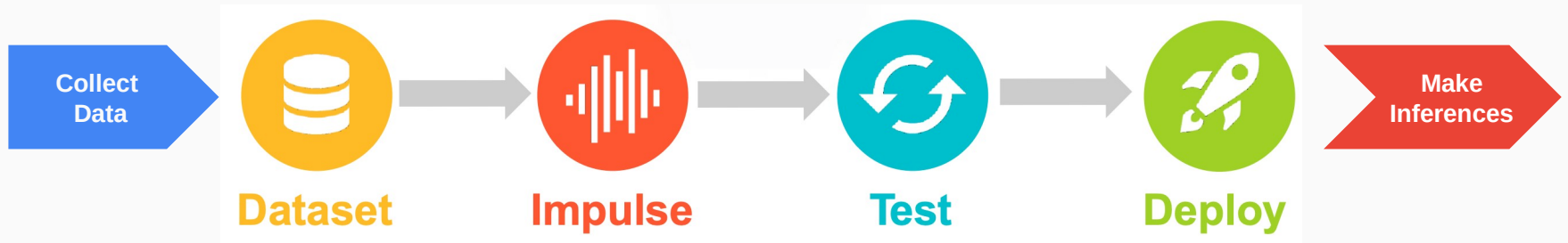


Machine Learning Workflow (“How”)

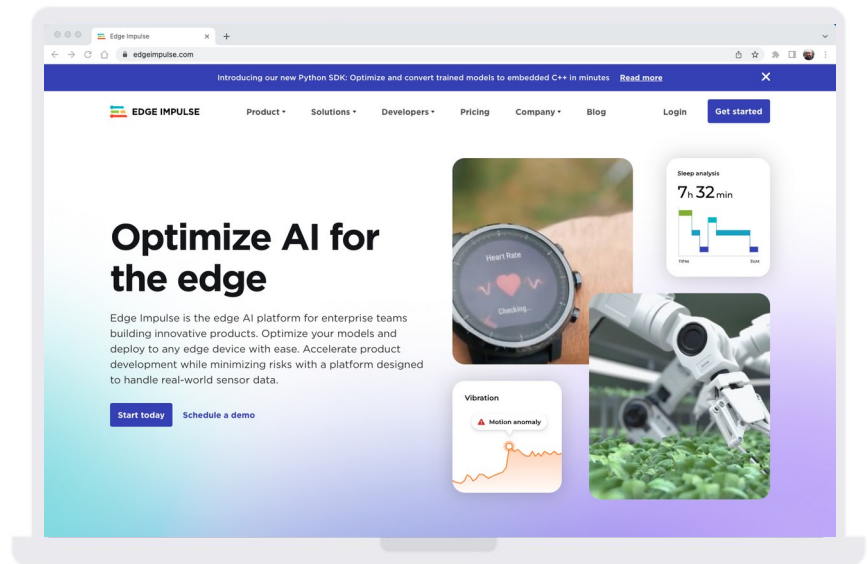
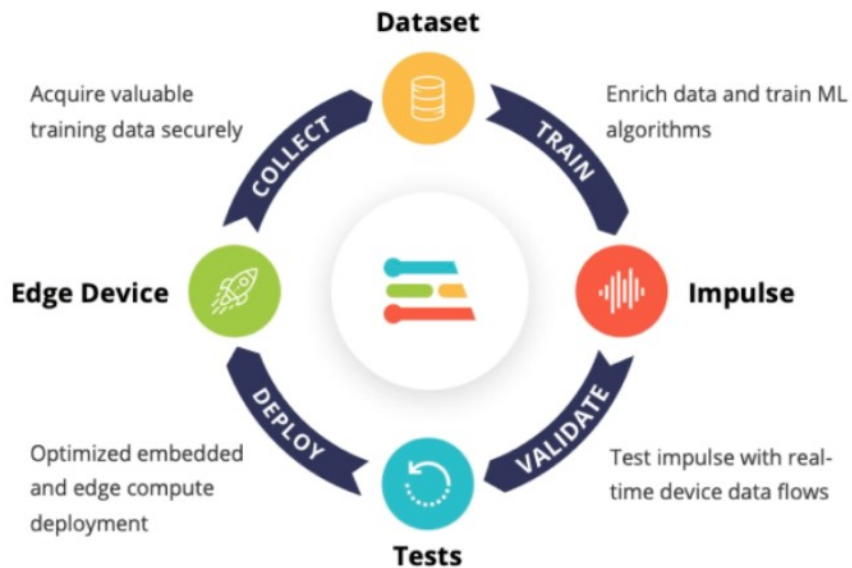


Machine Learning Workflow (“How”)





EI Studio - Embedded ML platform (“AutoML”)



Learn more at <http://edgeimpulse.com>



- Register a new account in Edge Impulse. Create a **new project** and give it a name:

Enter the name for your new project:

Choose your project type:

☒ **Personal**
20 min job limit, 4GB or 4 hours of data, limited collaboration.

☐ **Enterprise**
No job or data size limits, higher performance, custom blocks.

Choose your project setting:

☒  **Public**
Anyone on the internet can view and clone this project under the licence: [Apache 2.0](#). Only invited users will be able to edit.

☐  **Private (0 of 2 remaining)**
Only invited users can edit and view your project.
To request additional projects, [contact sales](#)

 Want full-feature access and unlimited projects? [Try Enterprise free.](#)

Create new project

- Once the project is created, select **Add Existing data** and then **Upload data**. Choose the files to be uploaded.
- Leave the default setting and make sure that Split between training and testing is selected.

Upload into category

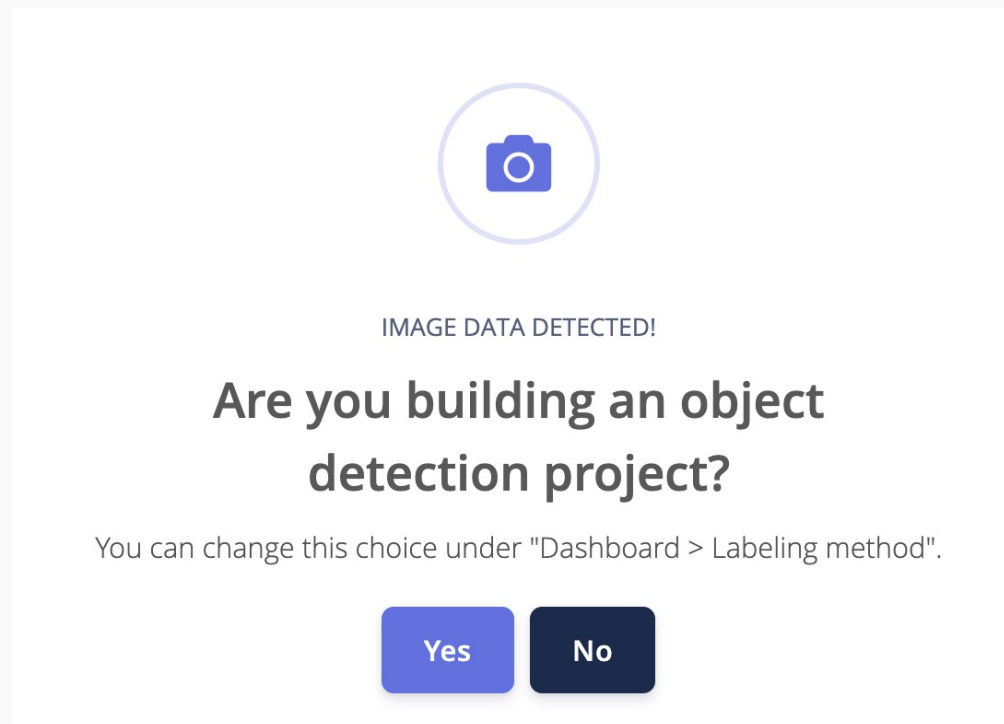
☒ Automatically split between training and testing ?

☐ Training

☐ Testing

- Click on Upload data on the lower right of the window.

- If you are working on an image classification project, make sure you click on No in the pop-up page:



- The uploaded samples will now appear in the Data acquisition section.

EDGE IMPULSE Marco / KIC-Mask

Dashboard Devices Data acquisition Impulse design Create impulse EON Tuner Retrain model Live classification Model testing Versioning Deployment

GETTING STARTED Documentation

Data acquisition Dataset Data explorer Data sources CSV Wizard

DATA COLLECTED
7,308 items

TRAIN / TEST SPLIT
80% / 20%

Collect data
[Connect a device](#) to start building your dataset.

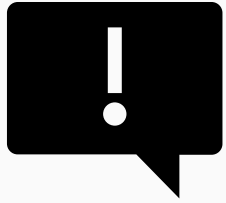
Dataset

Training (5,863) Test (1,445)

SAMPLE NAME	LABEL	ADDED	LENGTH
mask.42	mask	Today, 21:16:00	-
mask.45	mask	Today, 21:16:00	-
mask.8	mask	Today, 21:16:00	-
mask.48	mask	Today, 21:16:00	-
mask.6	mask	Today, 21:16:00	-
mask.4	mask	Today, 21:14:30	-

RAW DATA
mask.8

Metadata



It is important for the dataset to be balanced!

DATA COLLECTED
7,308 items

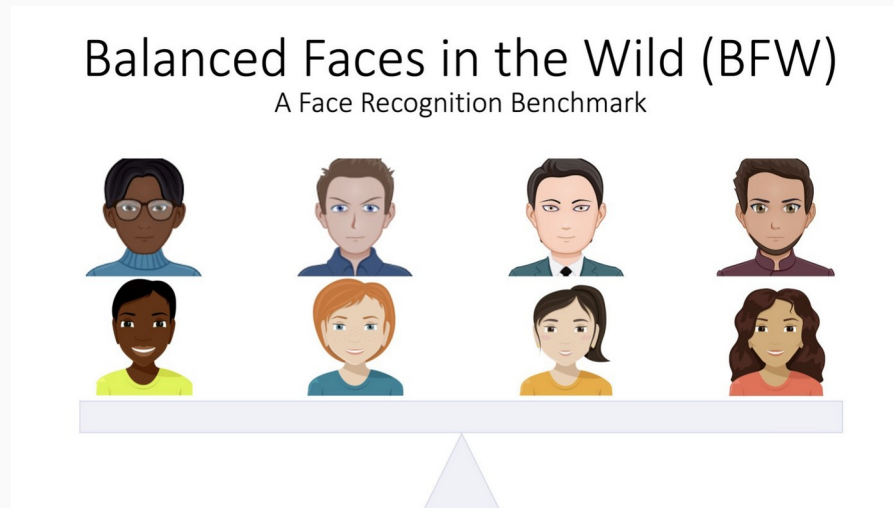


Balanced datasets

- Classification models attempt to categorize data into different buckets. In an imbalanced dataset, one bucket makes up a large portion of the training dataset (the majority class), while the other bucket is underrepresented in the dataset (the minority class). **The problem with a model trained on imbalanced data is that the model learns that it can achieve high accuracy by consistently predicting the majority class, even if recognizing the minority class is equal or more important when applying the model to a real-world scenario.**
- Consider the case of collecting training data for a model predicting medical conditions. Most of the patient data collected, let's say 95 percent, will likely fall into the healthy bucket, while the sick patients make up a much smaller portion of the data. During training, the classification model learns that it can achieve 95 percent accuracy if it predicts “healthy” for every piece of data it encounters. That's a huge problem because what doctors really want the model to do is identify those patients suffering from a medical condition.

Balanced datasets

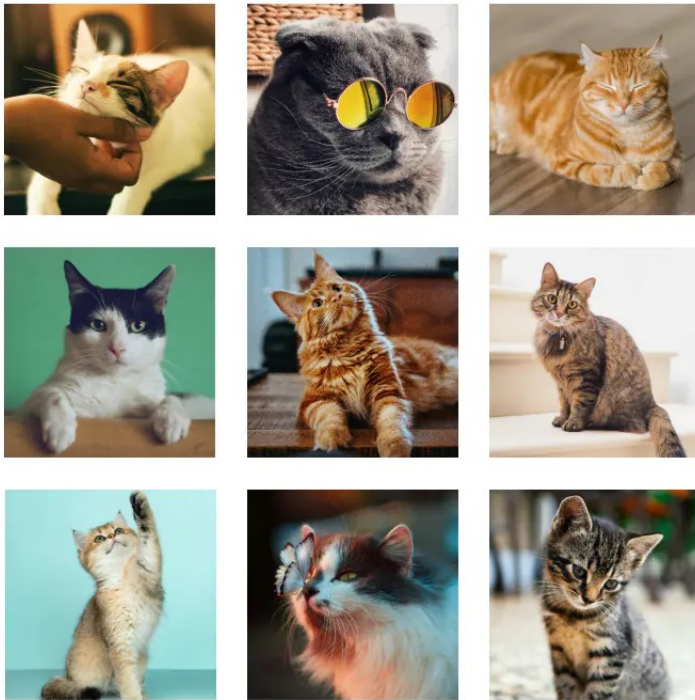
- **Imbalanced datasets** create challenges for predictive modeling, but they're a common and anticipated problem because the real world is full of imbalanced examples.



- **Balancing a dataset** makes training a model easier because it helps prevent the model from becoming biased towards one class. In other words, the model will no longer favor the majority class just because it contains more data.

Unbalanced Dataset

Cats



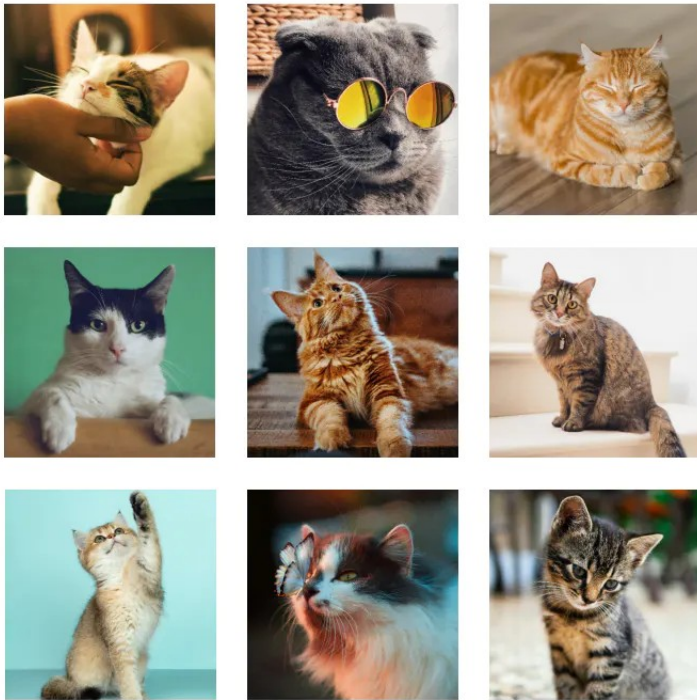
Dogs



Balanced datasets

Get More Data

Cats

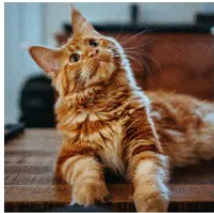


Dogs



Undersampled

Cats



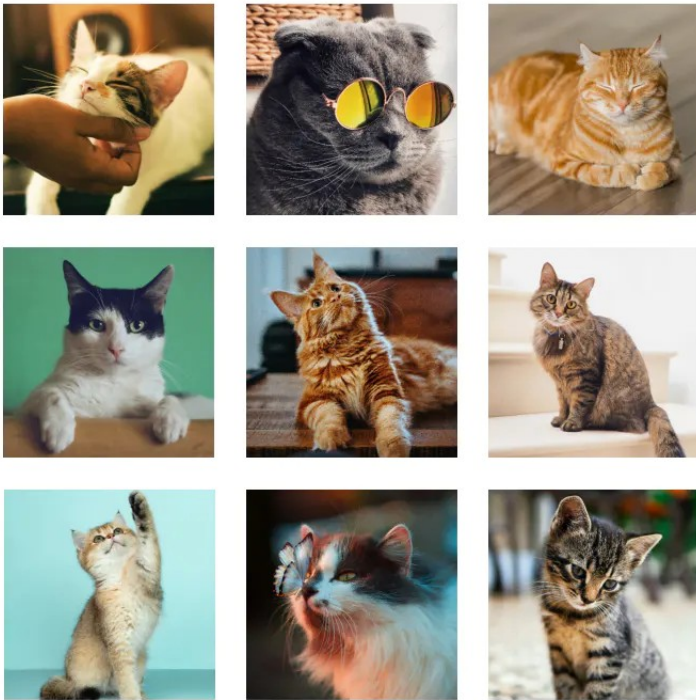
Dogs



Balanced datasets

Oversampled

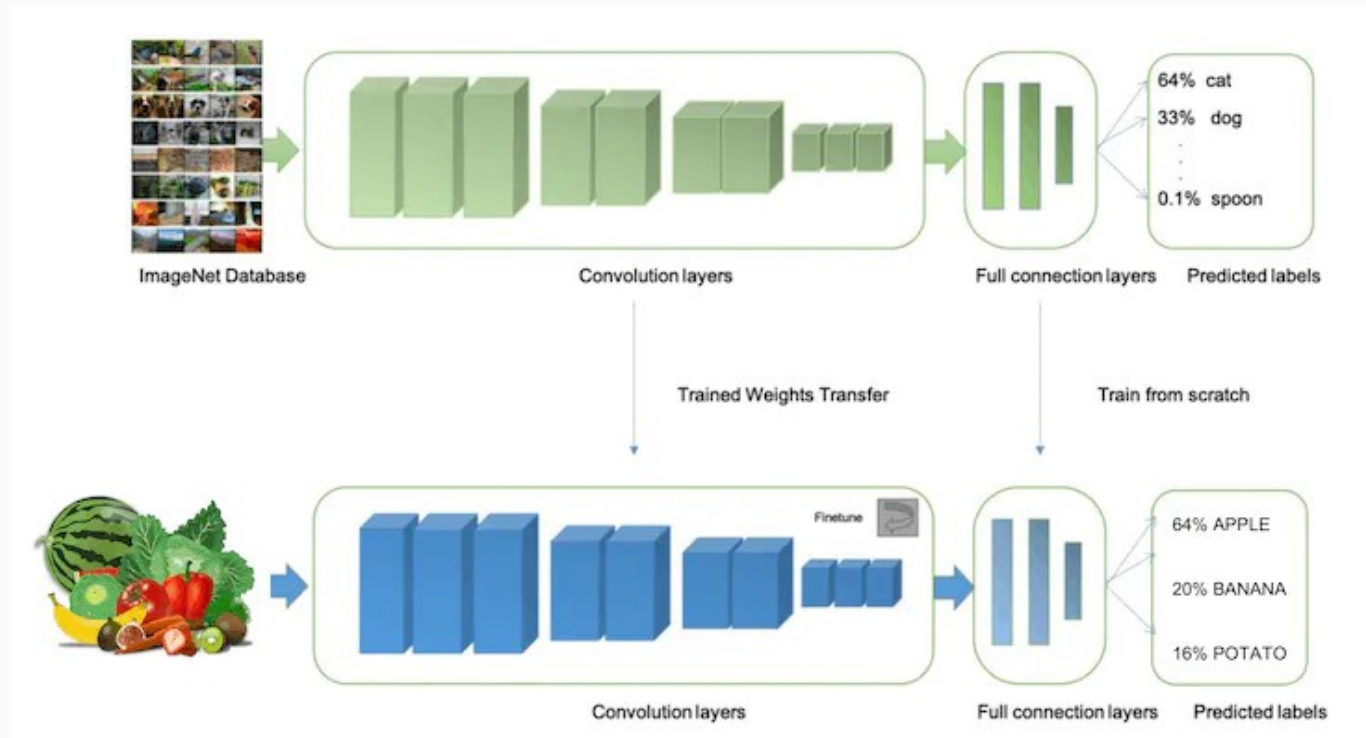
Cats



Dogs

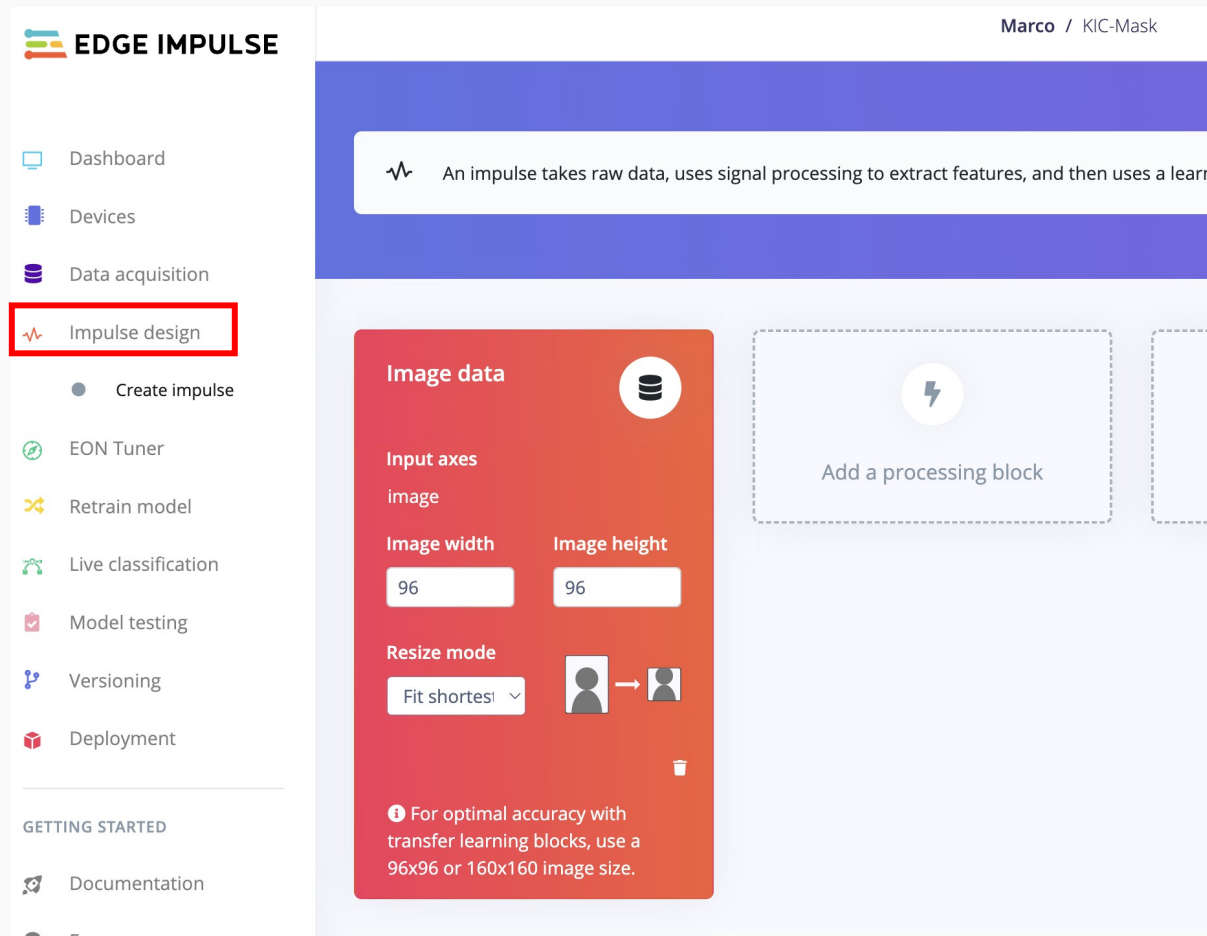


- An impulse takes raw data (in this case, images), extracts features (resize pictures), and then uses a learning block to classify new data.
- Classifying images is the most common use of Deep Learning, but much data should be used to accomplish this task. Do we have enough images for each category? Not at all! We will need thousand of images to train the model.
- We can solve this issue by re-training a previously trained model with thousands of images. We called this technique **Transfer Learning (TL)**.



With TL, we can fine-tune a pre-trained image classification model on our data, performing well even with relatively small image datasets (our case).

- Let's start creating the impulse. Click on **Impulse Design**.



The screenshot displays the Edge Impulse web application interface. On the left is a sidebar with the 'EDGE IMPULSE' logo at the top. Below the logo, a list of navigation items is shown: 'Dashboard', 'Devices', 'Data acquisition', 'Impulse design' (which is highlighted with a red rectangular box), 'Create impulse', 'EON Tuner', 'Retrain model', 'Live classification', 'Model testing', 'Versioning', and 'Deployment'. At the bottom of the sidebar is a 'GETTING STARTED' section with a 'Documentation' link. The main content area on the right has a header 'Marco / KIC-Mask'. Below the header is a blue banner with a waveform icon and the text 'An impulse takes raw data, uses signal processing to extract features, and then uses a learn'. The central part of the interface features a red 'Image data' block with a database icon, 'Input axes' set to 'image', 'Image width' and 'Image height' both set to '96', and a 'Resize mode' dropdown set to 'Fit shortest'. To the right of this block is a dashed box labeled 'Add a processing block' with a lightning bolt icon. At the bottom of the red block is a tip: 'For optimal accuracy with transfer learning blocks, use a 96x96 or 160x160 image size.'

- Starting from the raw images, we will resize them (96x96) pixels and use Transfer Learning.
- Click on Save Impulse.

Image data



Input axes

image

Image width

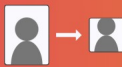
96

Image height

96

Resize mode

Fit shortest





 For optimal accuracy with transfer learning blocks, use a 96x96 or 160x160 image size.

Image



Name

Image

Input axes (1)

☒ image



Transfer Learning (Images)



Name

Transfer learning

Input features

☒ Image

Output features

2 (mask, nomask)



Output features



2 (mask, nomask)

Save Impulse

- By changing the data samples to Greyscale, we save a lot of memory. Each sample has now 96x96x1 features (216). Using RGB color depth, the dimension would be three times bigger. Working with Grayscale helps to reduce the amount of final memory needed for inference.
- Click on Image, on the left, to choose Greyscale. Click on Save parameters.

The screenshot shows a web interface for image processing. At the top, there's a 'Raw data' section with a 'Show:' dropdown set to 'All labels' and a 'nomask.1202 (nomask)' dropdown. Below this is a color image of a woman. The interface is divided into two main columns. The left column contains three sections: 'Raw features' with a hex color code, 'Parameters' with a 'Color depth' dropdown set to 'Grayscale', and a 'Save parameters' button. The right column contains two sections: 'DSP result' with a grayscale image of the woman, and 'Processed features' with a list of numerical values.

Raw data

Show: All labels

nomask.1202 (nomask)

Raw features

0xf100c, 0xf100b, 0xf100a, 0x11120a, 0x14120a, 0x17130a, 0x1c180f, 0x21...

Parameters

Image

Color depth ⓘ Grayscale

Save parameters

DSP result

Image

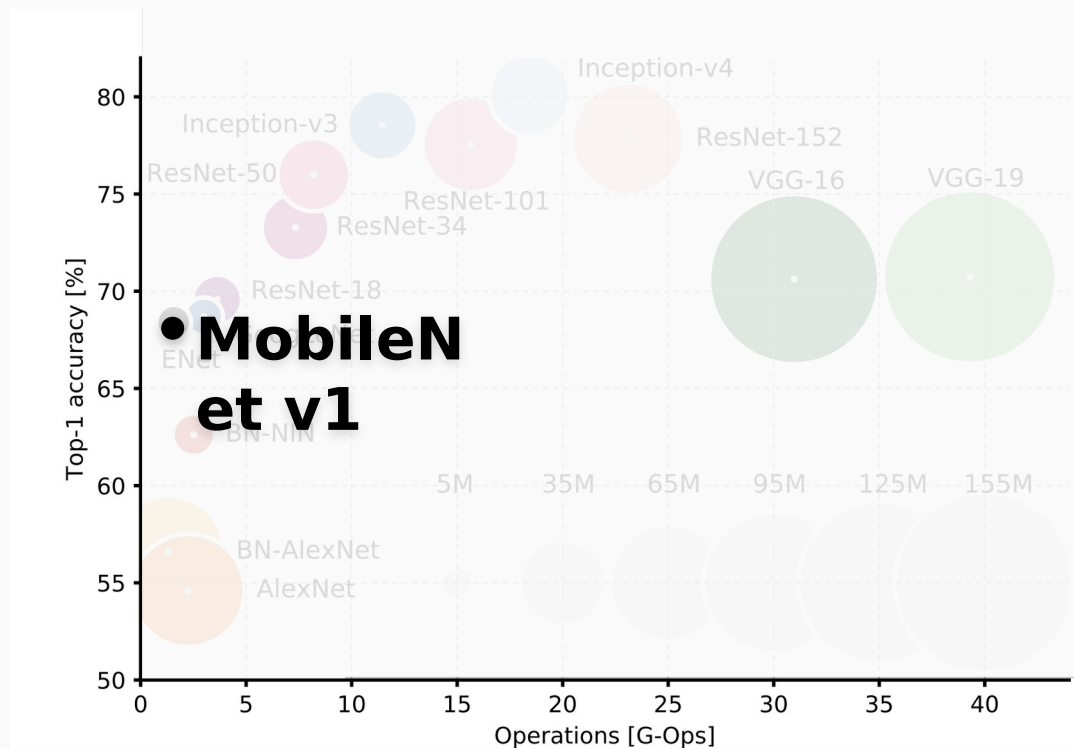
Processed features ⓘ

0.0598, 0.0593, 0.0589, 0.0658, 0.0694, 0.0752, 0.0948, 0.1066, 0.1188, ...

- In 2007, Google introduced **MobileNetV1**, a family of general-purpose computer vision neural networks designed with mobile devices in mind to support classification, detection, and more. MobileNets are small, low-latency, low-power models parameterized to meet the resource constraints of various use cases.
- Although the base MobileNet architecture is already tiny and has low latency, many times, a specific use case or application may require the model to be smaller and faster. MobileNet introduces a straightforward parameter α (alpha) called **width multiplier** to construct these smaller and less computationally expensive models. The role of the width multiplier α is to thin a network uniformly at each layer.

- Edge Impulse Studio offers MobileNet V1 (96x96 images) and V2 (96x96 and 160x160 images), with several different α values (from 0.05 to 1.0). For example, you will get the highest accuracy with V2, 160x160 images, and $\alpha=1.0$. Of course, there is a trade-off. The highest the accuracy, the more memory (around 1.3M RAM and 2.6M ROM) will be needed to run the model and imply more latency.
- The smaller footprint will be obtained at another extreme with **MobileNet V1** and $\alpha=0.10$ (around 53.2K RAM and 101K ROM).

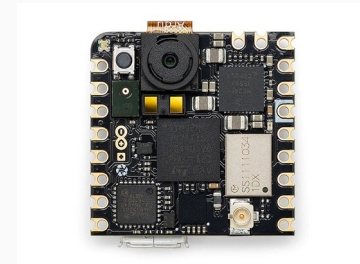
Model Evolution



MobileNet v1

Model	Size	Top-1 Accuracy
MobileNet v1	16 MB	0.713

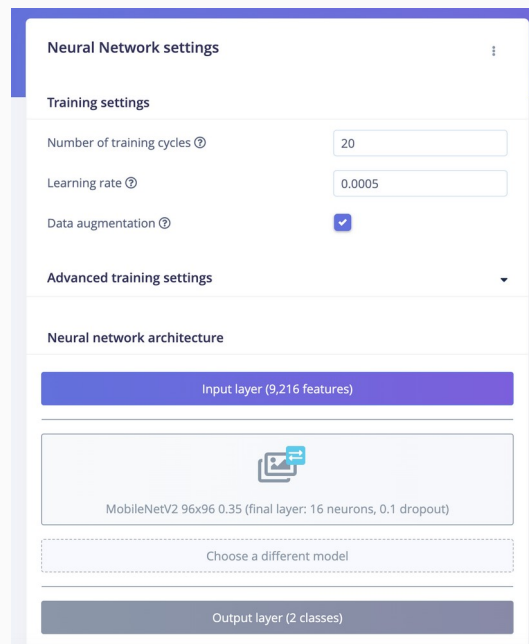
Fine for mobile phones with GB of RAM, but 64X microcontroller RAM



Our board only has **1MB** of RAM (memory)

Lab

- Another important technique to be used with Deep Learning is **Data Augmentation**. Data augmentation is a method that can help improve the accuracy of machine learning models, creating additional artificial data. A data augmentation system makes small, random changes to your training data during the training process (such as flipping, cropping, or rotating the images).
- Let's choose Transfer Learning on the left, and start with the default values (MobileNetV2, 96x96, 0.35). Don't forget to select **Data augmentation**.



The screenshot displays the 'Neural Network settings' interface. It is divided into several sections: 'Training settings' with input fields for 'Number of training cycles' (set to 20) and 'Learning rate' (set to 0.0005), and a checked checkbox for 'Data augmentation'; 'Advanced training settings' which is currently collapsed; 'Neural network architecture' showing a stack of layers: an 'Input layer (9,216 features)' in a blue box, a 'MobileNetV2 96x96 0.35 (final layer: 16 neurons, 0.1 dropout)' layer in a light blue box with a small icon, a dashed box labeled 'Choose a different model', and an 'Output layer (2 classes)' in a dark grey box.

- Before clicking on Start Training, choose on the top right side of the window the **Target device**. Click on Nicla Vision and then on **Start Training**.

 **Configure your target device and application budget** ×

Target device
Define your target device requirements to inform model optimizations and performance calculations. No device yet? Use the default settings which you can change at any time.

Target device

Arduino Nicla Vision (Cortex-M7 480MHz) ▼

Processor family

Cortex-M ▼

Clock rate ?

480 | MHz ▼
Max

Custom device name (optional) ?



- The result is great. Testing the model with the data put apart before training (Test Data), we got an accuracy of approximately 97%.



- Before deployment of the model on the Nicla device, it is possible to perform Live Classification using a smartphone. Go to the **Live Classification** section and click on **Connect a Development board**:

Marco / KWS-KIC

Connect a development board

Classify new data

Device ? No devices connected

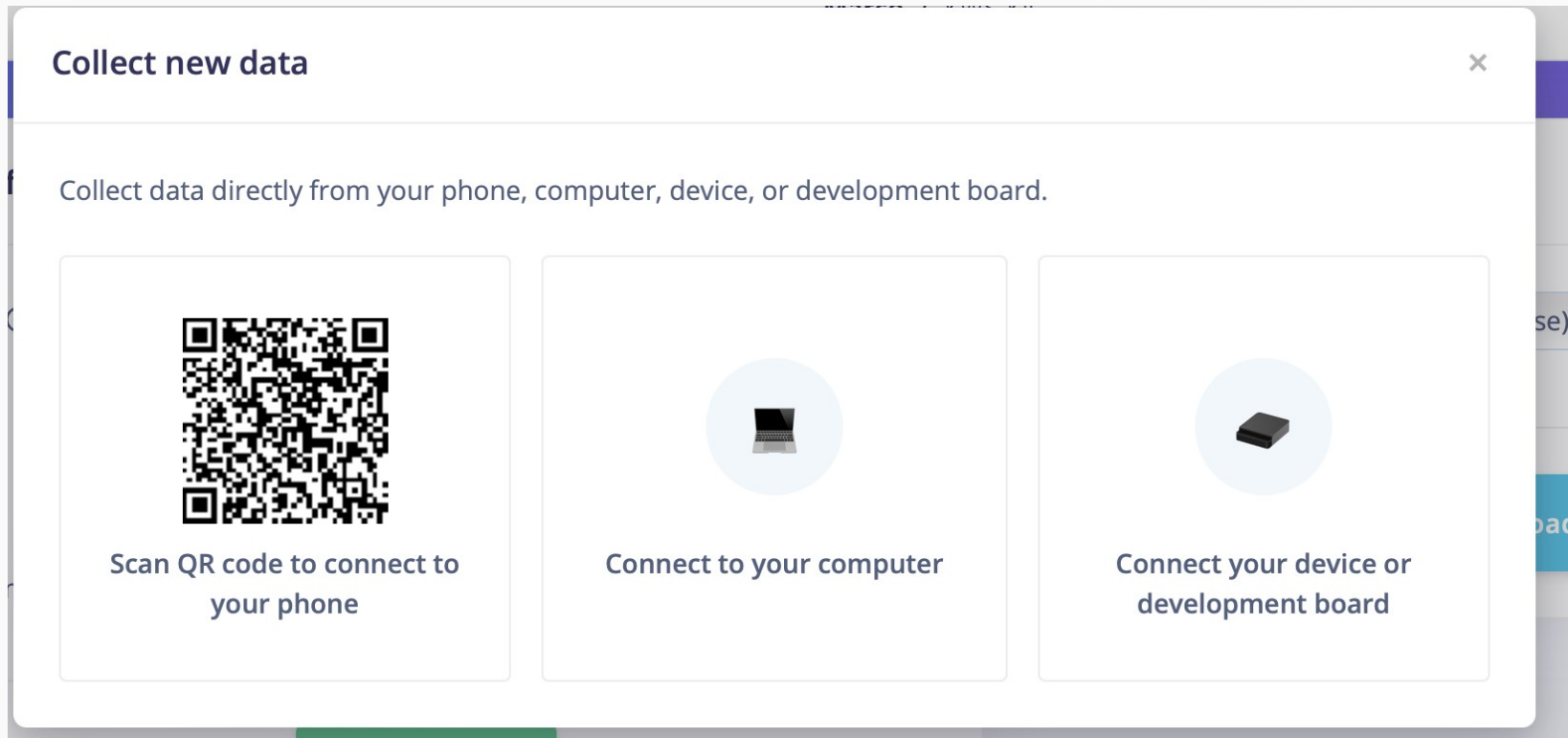
Sensor

Sample length (ms.) 5000

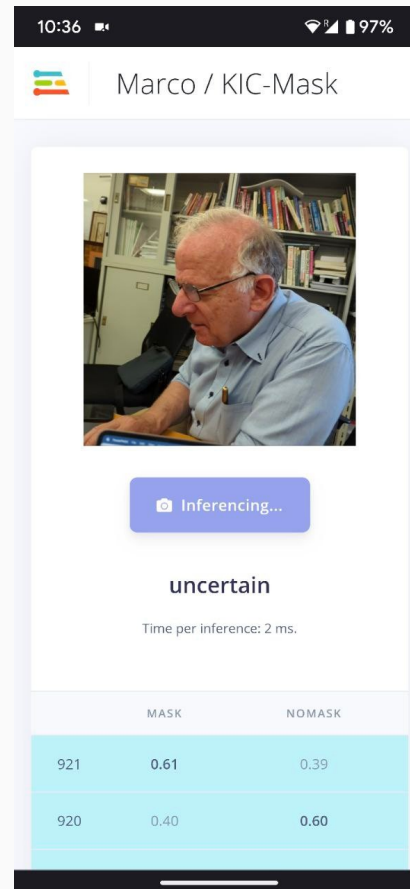
Classify existing data

noise.doing

- Point your phone to the QR code and select the link.



- Your phone will be connected to Edge Impulse Studio. Select the option **Classification** on the app, and when it is running, start testing your images, confirming that the model is working with live and real data:



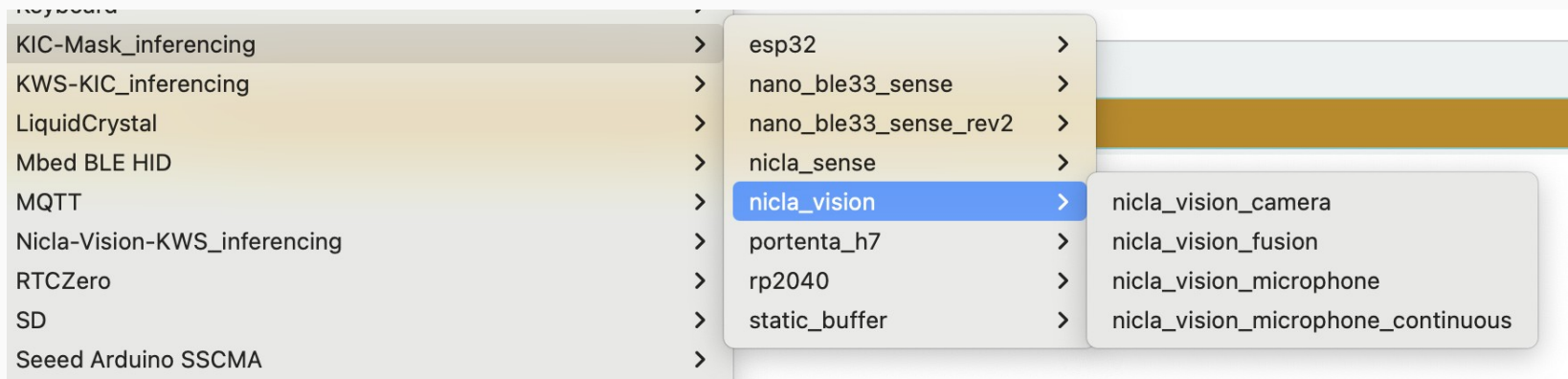
- To deploy the model on the Nicla, Edge Impulse Studio will package all the needed libraries, preprocessing functions, and trained models, downloading them to your computer. Select Deployment on the left side of the window.

The screenshot shows the Edge Impulse Studio interface. On the left, a sidebar contains a list of navigation items: Dashboard, Devices, Data acquisition, Impulse design, Create impulse, MFCC, Classifier, EON Tuner, Retrain model, Live classification, Model testing, Performance calibration, Versioning, and Deployment. The 'Deployment' item is highlighted with a red rectangle. The main area displays the 'Configure your deployment' page. It includes a search bar for deployment options, a section for 'DEFAULT DEPLOYMENT' featuring the 'C++ library' (described as a portable C++ library with no external dependencies), and a section for 'MODEL OPTIMIZATIONS'. In the optimizations section, the 'Enable EON™ Compiler' toggle is turned on, with a note that it maintains the same accuracy while reducing memory by up to 50%. Below this is a table showing performance metrics for quantized (int8) models.

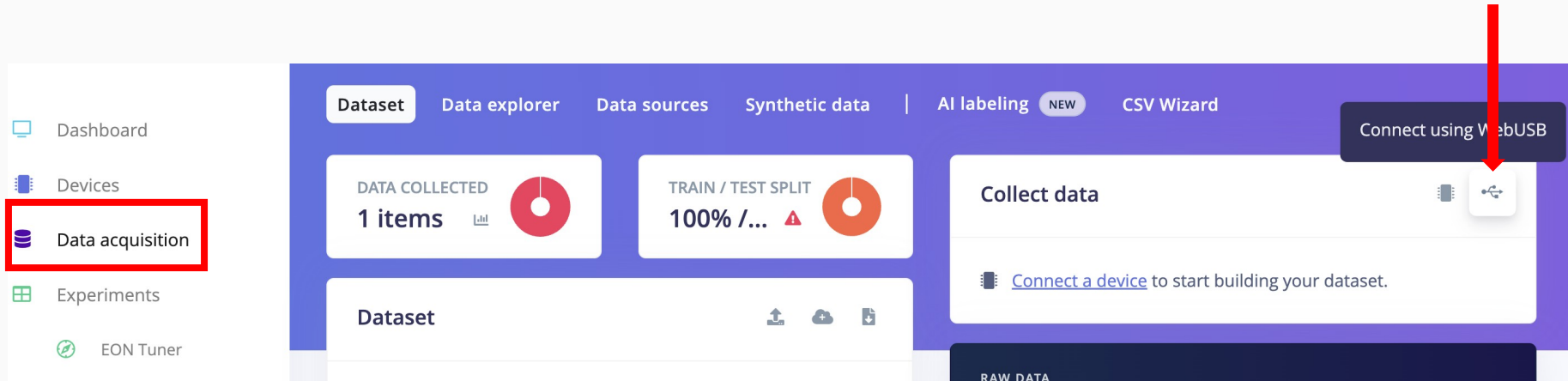
	MFCC	CLASSIFIER	TOTAL
LATENCY	176 ms.	3 ms.	179 ms.
RAM	13.0K	3.7K	13.0K
FLASH	-	26.9K	-
ACCURACY	-	-	-

- You should select the option **Arduino Library** and at the bottom, select **Quantized (Int8)**, de-select the option **Enable EON Compiler**. You should select the target device (lowest part of the screen). You can now press the button Build.
- When the Build button is pressed, a Zip file will be created and downloaded to your computer.
- On your Arduino IDE, go to the **Sketch** tab and select the option **Include Library → Add .ZIP Library**, and Choose the new .zip file you have downloaded.

- In your Arduino IDE, go to the File → Examples tab and look for your project, and select Nicla Vision:

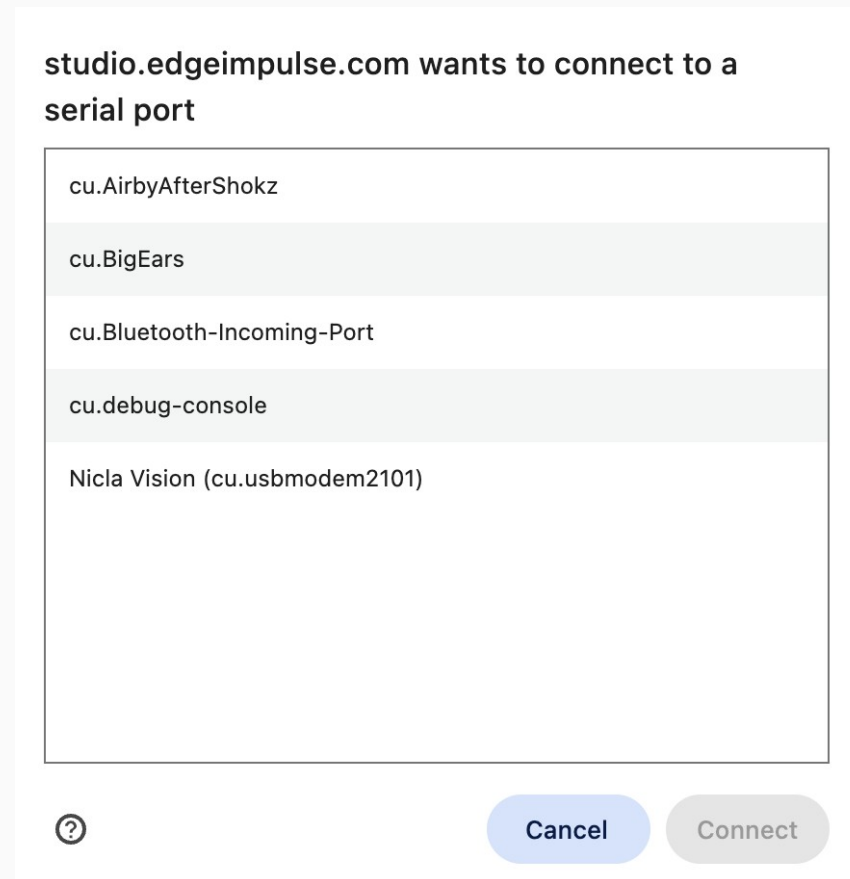


- If you want to collect images using the Nicla directly, follow these steps:



Make sure you are using Chrome as WebUSB is only available in Chrome!

- Select the USB port where the Nicla is connected:



- On the right you will see the live camera feed and you will be able to assign a label to the images you collect. Click on Start sampling to save an image.

Collect data

Device ?

51:0D:39:31:35:32

Label

smile

Sensor

Camera (96x96)

Camera feed



Start sampling

Thanks!

TinyML whispers low,
AI in palms, humble glow,
Minds bloom, small yet grow.

mzennaro@ictp.it